由浅入深的 Cython 逆向指南

—— c10udInk.

About Us

山石网科安全技术研究院

>>> 研究方向包括WEB安全、移动安全、智能安全和信创安全等，参编过多份安全标准。

>>> 曾为上合峰会、财富论坛、港珠澳大桥的开通等重大活动提供了网络安保工作并获得感谢信。

>>> 获得过苹果、红帽等知名企业的官网致谢，连续多年进入微软全球最具价值安全研究员榜单，公布过多个具有全球影响力的严重漏洞，在 Blackhat、Defcon、HITB、CanSecWest 等国际安全大会分享过研究成果，获得了诸多 CVE 漏洞编号。

>>> 荣获过红帽杯企业组冠军，补天杯最具价值漏洞奖，强网拟态防御国际精英挑战赛一等奖，第五空间网络安全大赛特等奖，津门杯国际网络安全创新大赛一等奖，陇警杯网络安全大赛第一名等优异成绩。

About Me

REer in [NEURON, S1uM4i, Sloth]

>>> **y[2020] =** 首次接触 Python 字节码，从 Python 不熟练到手撕字节码一天速成。

>>> **y[2021:2023] =** \

. . . 研究的魔改 pyc 越来越多，能直接被反编译的都是时代的眼泪；

. . . 分析了 Python 字节码隐写工具 stegosaurus，在版本不兼容的情况下手动抠出一个 flag，赛后做了一些魔改使其兼容 Python 3.10（https://github.com/c10udlnk/stegosaurus）；

. . . Python 逆向开始卷到了 Cython，静态分析慢慢上手，结合动态交互开始能解出 flag 了。

>>> **y[2023] =** 毕设选题做的是 Python 代码保护相关，深入研究了 Oxyry、pyc_obscure、Py***or 等混淆保护，设计了一套加密器。

>>> **y[2024] =** 结合 xasm 实现对手撕字节码的自动化（https://github.com/c10udlnk/dis2xasm）。

>>> 目前正在研究 Cython 逆向的自动化（https://github.com/c10udlnk/cythonHelper）。

>>> # 吃灰博客：https://c10udlnk.top/

初探 Cython 及 Cython 逆向

Cython 是什么?

和 Python、CPython 的关系

CTF 逆向中的 Cython

关于 Cython

>>> https://cython.org/

>>> https://github.com/cython/cython

>>> 一个基于 Pyrex 的 Python 编译器，它使**为 Python 编写 C 扩展**就像编写 Python 本身一样简单。

>>> 将代码段从动态 Python 语义移动到静态且快速的 C 语义，并可以直接操作外部库中定义的类型。

>>> 源代码被转换为优化的 C/C++ 代码并编译为 Python 扩展模块，从而加快了程序的执行速度，并可与外部 C 库紧密集成，同时保持 Python 语言的编写简易性。

>>> # 关于 Python 的 C 扩展：https://docs.python.org/zh-cn/3/extending/extending.html

>>> #“C 扩展接口特指 CPython，扩展模块无法在其他 Python 版本上工作。在大多数情况下，应该避免写 C 扩展，来保持可移植性。举个例子，如果你的用例调用了 C 库或系统调用，你应该考虑使用 ctypes 模块或 cffi 库，而不是自己写 C 代码。”

编写 Python 的 C 扩展

>>> 添加 C 函数到扩展模块：

spam.py

```python
import os

def system(command):
    return os.system(command)
```

```c
static PyObject *
spam_system(PyObject *self, PyObject *args)
{
    const char *command;
    int sts;

    if (!PyArg_ParseTuple(args, "s", &command))
        return NULL;
    sts = system(command);
    return PyLong_FromLong(sts);
}
```

>>> 在模块的 method table 中添加对外调用接口：

```c
static PyMethodDef SpamMethods[] = {
    ...
    {"system",  spam_system, METH_VARARGS,
     "Execute a shell command."},
    ...
    {NULL, NULL, 0, NULL}        /* Sentinel */
};
```

>>> 在模块的定义结构中引用，并创建 C 扩展的 Entry Point：

```c
static struct PyModuleDef spammodule = {
    PyModuleDef_HEAD_INIT,
    "spam",   /* name of module */
    spam_doc, /* module documentation, may be NULL */
    -1,       /* size of per-interpreter state of the module,
                 or -1 if the module keeps state in global variables. */
    SpamMethods
};
```

```c
PyMODINIT_FUNC
PyInit_spam(void)
{
    return PyModule_Create(&spammodule);
}
```

用 Cython 编写 Python 的 C 扩展

>>> 直接用 Python 编写

spam.py

```python
import os

def system(command):
    return os.system(command)
```

>>> cythonize 一把梭

```
└─○ cythonize -i spam.py
Compiling /home/c10udlnk/pr0bs/cython_test/c1/spam.py
[1/1] Cythonizing /home/c10udlnk/pr0bs/cython_test/c1/
/home/c10udlnk/.local/lib/python3.10/site-packages/Cyt
```

>>> 然后就拥有了对应的 C 扩展文件和编译好的库

```
└─○ ls | grep spam
spam.c
spam.cpython-310-x86_64-linux-gnu.so
spam.py
```

```
2400   __Pyx_XDECREF(__pyx_r);
2401   __Pyx_GetModuleGlobalName(__pyx_t_2, __pyx_n_s_os); if (unlikely(!__pyx_t_2)) __PYX_ERR(0, 4, __pyx_L1_
2402   __Pyx_GOTREF(__pyx_t_2);
2403   __pyx_t_3 = __Pyx_PyObject_GetAttrStr(__pyx_t_2, __pyx_n_s_system); if (unlikely(!__pyx_t_3)) __PYX_ERR
2404   __Pyx_GOTREF(__pyx_t_3);
2405   __Pyx_DECREF(__pyx_t_2); __pyx_t_2 = 0;
2406   __pyx_t_2 = NULL;
2407   __pyx_t_4 = 0;
2408   #if CYTHON_UNPACK_METHODS
2409   if (unlikely(PyMethod_Check(__pyx_t_3))) {
2410     __pyx_t_2 = PyMethod_GET_SELF(__pyx_t_3);
2411     if (likely(__pyx_t_2)) {
2412       PyObject* function = PyMethod_GET_FUNCTION(__pyx_t_3);
2413       __Pyx_INCREF(__pyx_t_2);
2414       __Pyx_INCREF(function);
2415       __Pyx_DECREF_SET(__pyx_t_3, function);
2416       __pyx_t_4 = 1;
2417     }
2418   }
2419   #endif
2420   {
2421     PyObject *__pyx_callargs[2] = {__pyx_t_2, __pyx_v_command};
2422     __pyx_t_1 = __Pyx_PyObject_FastCall(__pyx_t_3, __pyx_callargs+1-__pyx_t_4, 1+__pyx_t_4);
2423     __Pyx_XDECREF(__pyx_t_2); __pyx_t_2 = 0;
2424     if (unlikely(!__pyx_t_1)) __PYX_ERR(0, 4, __pyx_L1_error)
2425     __Pyx_GOTREF(__pyx_t_1);
2426     __Pyx_DECREF(__pyx_t_3); __pyx_t_3 = 0;
2427   }
```

Cython? Python? CPython?

>>> **Cython** = 一个可以将 Python 转化为 C 的编译器，也是一种语言（Python 的超集）

>>> **Python** = 一种众所周知的编程语言，可以让你快速工作并更有效地集成系统。

>>> **CPython** = Python 解释器的源码，并给用户提供 Python–C API 支持以编写 C 扩展

... # (https://github.com/python/cpython)

Python in CTF Reverse

>>> 在源码层面混淆 Python 代码或只提供字节码文本

>>> .pyc 字节码文件 # pyc反编译

>>> 魔改的 .pyc 字节码文件 # 修改魔改部分，反编译

. . . 修改或删除文件头

. . . co_code 中插入花指令

. . . 在冗余字节中嵌入密钥

. . . 魔改 Python 解释器 + 仅可用该解释器运行的代码

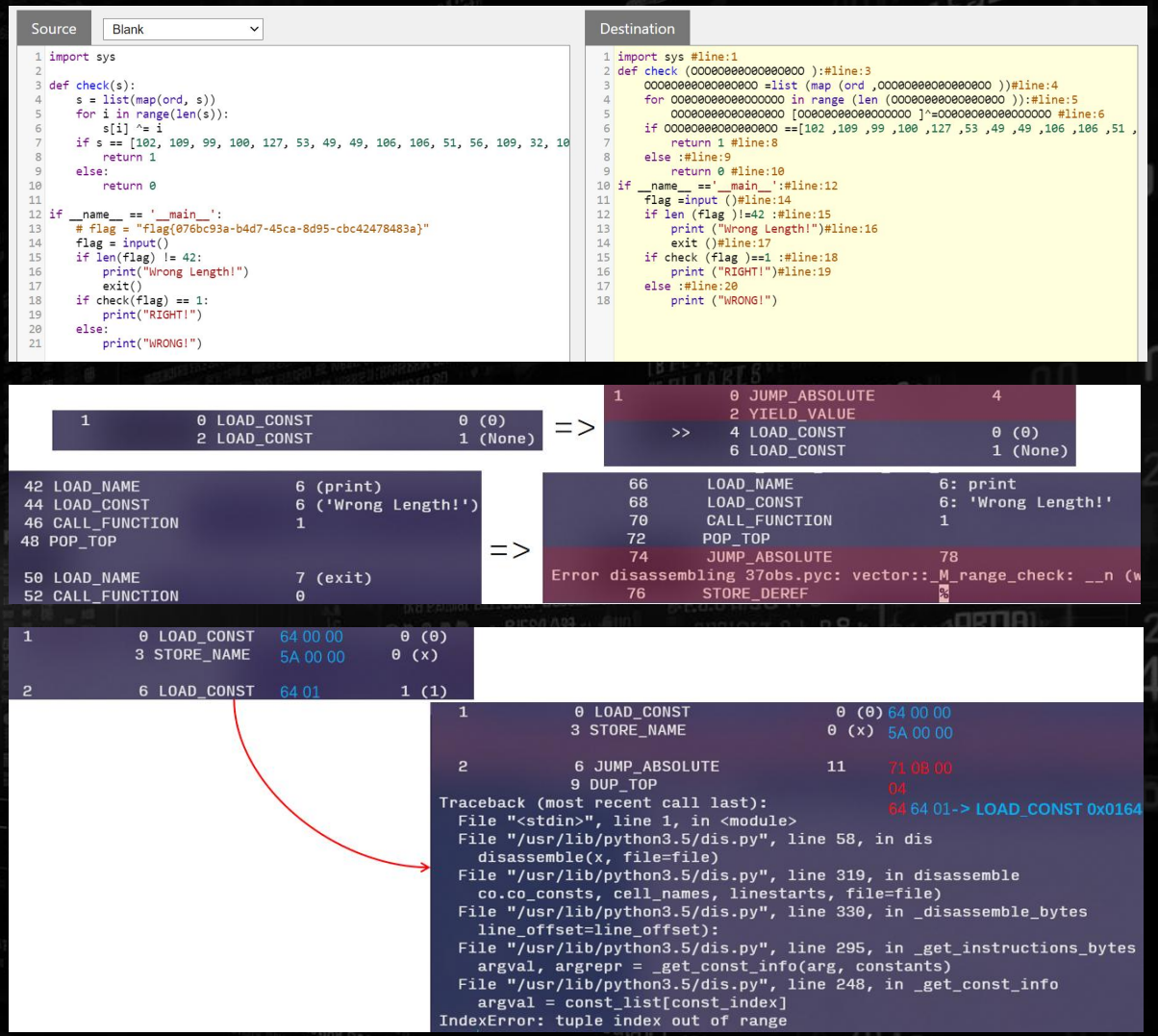
>>> Pyinstaller 打包 # 解包，pyc 反编译

>>> **cython 编译的扩展（.pyd/.so）# 人工手撕+动态**

. . . 给 main 函数，扩展中写入容易猜的加密（如 xor）

. . . main 函数中只写调用逻辑，全部加密都在扩展中

. . . # **Pyinstaller 打包**时对 import 进来的 Python 代码都会调用 Cython 来编译以加快调用速度；或者**单给扩展和代码**。



Cython 逆向之
动态取巧与静态硬看

动态 import，摸清大概思路

静态分析，补充函数细节

动静结合，将 flag 收入囊中

通过动态交互来取巧解题

>>> 首先判断给的扩展的文件格式和对应 Python 版本。

... # 有些题目看文件名就知道这些信息了，但大部分题目会改掉文件名

```
└─○ file chal.cpython-310-x86_64-linux-gnu.so
chal.cpython-310-x86_64-linux-gnu.so: ELF 64-bit LSB shared object, x86-64, vers
ion 1 (SYSV), dynamically linked, BuildID[sha1]=0bc77261ef0864da954addb236a92fa8
295c40ad, with debug_info, not stripped
```

... 可见是 Linux 中编译的扩展，x86-64。

```
└─○ strings chal.cpython-310-x86_64-linux-gnu.so | grep python
/usr/local/include/python3.10
/usr/local/include/python3.10/cpython
```

... Python 版本为 3.10。

通过动态交互来取巧解题

>>> 我们使用**对应平台和架构的环境**，运行**对应版本的 Python**，即可成功导入该模块。

... 使用 **dir** 可以看到该模块对外的函数接口和全局变量：

```
>>> import chal
>>> dir(chal)
['__builtins__', '__doc__', '__file__', '__loader__', '__name__', '__package__',
 '__spec__', '__test__', 'checkFlag', 'enc', 'k']
>>> chal.checkFlag
<cyfunction checkFlag at 0x4001f79f20>
>>> chal.enc
<cyfunction enc at 0x4001f79ff0>
>>> chal.k
[84, 104, 105, 115, 73, 115, 65, 75, 101, 121, 33, 33, 33, 33, 33, 33]
```

... **k** 是全局变量，可以轻松看到它的值。

... 而 **checkFlag** 和 **enc** 这两个函数由于是 cyfunction，所以没办法使用类似 **dis** 等模块套出它们的字节码，因为它们这类在 C 层面上编写的函数根本就没有 Python 字节码 :)

```
>>> chal.checkFlag.__code__.co_code
b''
>>> chal.enc.__code__.co_code
b''
```

通过动态交互来取巧解题

>>> 不过，我们还有其他方法可以旁敲侧击函数的一些性质。

. . . 比如 **co_varnames** 里包含其用到的变量名（参数+局部变量），**co_argcount** 表示其参数个数，与 **co_varnames** 结合就知道这个函数的参数名和用到的局部变量名了。

```
>>> chal.enc.__code__.co_varnames
('a', 'rslt', 'i')
>>> chal.enc.__code__.co_argcount
1
```

```
>>> chal.checkFlag.__code__.co_varnames
('flag',)
>>> chal.checkFlag.__code__.co_argcount
0
```

>>> 当然，我们还可以实际调用这两个函数，来测试其功能，甚至还有一些可以通过触发报错来探测逻辑：

```
>>> chal.enc(1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "chal.py", line 6, in chal.enc
    rslt.append(ord(a[i]) ^ k[i])
TypeError: 'int' object is not subscriptable
```

. . . 一眼 xor 了属于是。 #当然一些题目会 ban Traceback

通过动态交互来取巧解题

>>> 假设没有 Traceback，只有报错类型，同样也能测试出一些细节。

```
>>> chal.enc(1)
TypeError: 'int' object is not subscriptable
```

... 说明传入的参数是个**可切片的数据类型**，比如字符串或者列表。

```
>>> chal.enc([1, 2])
TypeError: ord() expected string of length 1, but int found
```

... 说明参数需要是 **char**。

```
>>> chal.enc("aaabc")
IndexError: string index out of range
```

... 说明参数**长度不够**，逐个长度爆破直到不报错为止。

```
>>> chal.enc("aaabcaaaaaaaaaa")
IndexError: string index out of range
>>> chal.enc("aaabcaaaaaaaaaaa")
[53, 9, 8, 17, 42, 18, 32, 42, 4, 24, 64, 64, 64, 64, 64, 64]
```

... 现在知道了参数 a 的一些信息：需要**长度为16的字符串**。

通过动态交互来取巧解题

>>> 我们前面输出的 **k** 刚好也是 16 字节，那首先可以猜一些**一一对应的常见加密方法**，比如 xor；或者**密钥块和明文块的长度都为 16** 的对称密码算法，比如 AES、SM4 等，或**使用了 ECB 加密模式（没有 iv）**的 IDEA、Blowfish、TEA 系列等，或**密钥长度和明文长度不定的流密码**如 RC4 等。 # RE 主打一个猜

. . . 总之一种种加密试过去，在该题**被解出的次数多/难度不大/比赛不难**的情况下很容易猜出来。

```
>>> a = "aaabcaaaaaaaaaaa"
>>> chal.k
[84, 104, 105, 115, 73, 115, 65, 75, 101, 121, 33, 33, 33, 33, 33, 33]
>>> chal.enc(a)
[53, 9, 8, 17, 42, 18, 32, 42, 4, 24, 64, 64, 64, 64, 64, 64]
>>> [ord(a[i]) ^ chal.k[i] for i in range(16)]
[53, 9, 8, 17, 42, 18, 32, 42, 4, 24, 64, 64, 64, 64, 64, 64]
```

通过动态交互来取巧解题 + 是时候来点静态分析了

>>> 那么该题只剩被比对的数组没有获取到了。

>>> 常量在动态交互中获取不到，反编译 .so 文件试试（以 ghidra 为例）：

. . . 可以看到这里有一个 **plVar11 = (long *)PyList_New(0x10);**，开了一个长度为 16 的列表，而后有一系列赋值过程，这个很有可能就是我们要的数组。

. . . 这些全局变量可以通过找交叉引用找到它们代表的 PyObject：

. . . **DAT_0010cad8 = PyLong_FromLong(0x6d)**，代表 **DAT_0010cad8** 这个变量其实是 **0x6d** 的PyObject，可以理解为 **plVar11[0] = 0x6d**，以此类推可以拿到这个被比对数组。

. . . # pplVar1 = (long **)plVar11[3]

. . . # = plVar11.ob_item

```
if ((((((((iVar4 < 0) || (DAT_0010ca40 = PyLong_FromLong(0xf), DAT_0010ca40 == 0)
         ) || (DAT_0010ca48 = PyLong_FromLong(0x10), DAT_0010ca48 == 0)) ||
       ((DAT_0010ca50 = PyLong_FromLong(0x11), DAT_0010ca50 == 0 ||
        (DAT_0010ca58 = PyLong_FromLong(0x12), DAT_0010ca58 == 0)))) ||
      (DAT_0010ca60 = PyLong_FromLong(0x13), DAT_0010ca60 == 0)) ||
     (((DAT_0010ca68 = PyLong_FromLong(0x18), DAT_0010ca68 == 0 ||
       (DAT_0010ca70 = (long *)PyLong_FromLong(0x21), DAT_0010ca70 == (long *)0x0
       )) || ((DAT_0010ca78 = PyLong_FromLong(0x23), DAT_0010ca78 == 0 ||
              (((DAT_0010ca80 = PyLong_FromLong(0x2a), DAT_0010ca80 == 0 ||
                (DAT_0010ca88 = (long *)PyLong_FromLong(0x41),
                DAT_0010ca88 == (long *)0x0)) ||
               (DAT_0010ca90 = PyLong_FromLong(0x43), DAT_0010ca90 == 0)))))))
    || (((DAT_0010ca98 = (long *)PyLong_FromLong(0x49),
         DAT_0010ca98 == (long *)0x0 ||
         (DAT_0010caa0 = (long *)PyLong_FromLong(0x4b),
         DAT_0010caa0 == (long *)0x0)) ||
        ((DAT_0010caa8 = (long *)PyLong_FromLong(0x54),
         DAT_0010caa8 == (long *)0x0 ||
         (((DAT_0010cab0 = PyLong_FromLong(0x57), DAT_0010cab0 == 0 ||
           (DAT_0010cab8 = PyLong_FromLong(0x58), DAT_0010cab8 == 0)) ||
          ((DAT_0010cac0 = (long *)PyLong_FromLong(0x65),
           DAT_0010cac0 == (long *)0x0 ||
           (((DAT_0010cac8 = (long *)PyLong_FromLong(0x68),
             DAT_0010cac8 == (long *)0x0 ||
             (DAT_0010cad0 = (long *)PyLong_FromLong(0x69),
             DAT_0010cad0 == (long *)0x0)) ||
            (DAT_0010cad8 = PyLong_FromLong(0x6d), DAT_0010cad8 == 0)))))))))
    || ((DAT_0010cae0 = PyLong_FromLong(0x72), DAT_0010cae0 == 0 ||
        (DAT_0010cae8 = (long *)PyLong_FromLong(0x73), DAT_0010cae8 == (long *)0x0
        )))) ||
   (DAT_0010caf0 = (long *)PyLong_FromLong(0x79), DAT_0010caf0 == (long *)0x0)) {
```

```
212     plVar11 = (long *)PyList_New(0x10);
213     plVar12 = DAT_0010cad8;
214     if (plVar11 == (long *)0x0) {
215       *plVar10 = *plVar10 + -1;
216       if (*plVar10 == 0) {
217         _Py_Dealloc(plVar10);
218       }
219       uVar17 = 0xaba;
220       uVar18 = 0xb;
221     }
222     else {
223       pplVar1 = (long **)plVar11[3];
224       *DAT_0010cad8 = *DAT_0010cad8 + 1;
225       *pplVar1 = plVar12;
226       plVar12 = DAT_0010cab8;
227       *DAT_0010cab8 = *DAT_0010cab8 + 1;
228       pplVar1[1] = plVar12;
229       plVar12 = DAT_0010ca40;
230       *DAT_0010ca40 = *DAT_0010ca40 + 1;
231       pplVar1[2] = plVar12;
232       plVar12 = DAT_0010ca48;
233       *DAT_0010ca48 = *DAT_0010ca48 + 1;
234       pplVar1[3] = plVar12;
235       plVar12 = DAT_0010ca80;
236       *DAT_0010ca80 = *DAT_0010ca80 + 1;
237       pplVar1[4] = plVar12;
238       plVar12 = DAT_0010ca58;
239       *DAT_0010ca58 = *DAT_0010ca58 + 1;
240       pplVar1[5] = plVar12;
241       plVar12 = DAT_0010ca78;
242       *DAT_0010ca78 = *DAT_0010ca78 + 1;
243       pplVar1[6] = plVar12;
244       plVar12 = DAT_0010cae0;
```

动态交互还有——

>>> Hook 大法好！

>>> 可以将任何自定义的函数代替扩展里原来的函数。

... 定义一个函数 **hook_func**，尽量还原该函数原本功能（不影响后续处理），插入 **print**；

... 使用 **MOD.__dict__["func"] = hook_func** 将自定义函数挂进去；

... 调用函数，get 中间值。

>>> 函数越多越细，接口越多，就能获取越多的中间值。

```
>>> import Challenge as c
>>> dir(c)
['__builtins__', '__doc__', '__file__', '__loader__', '__name__', '__pa
nc_a', 'i2b', 'keyExpend']
>>> def hook_func3a(a, b):
...     print(a, b)
...     mul = a * b
...     hd = (mul >> 16) & 0xFFFF
...     ld = mul & 0xFFFF
...     if ld < hd:
...         return ld - hd + 1
...     else:
...         return ld - hd
...
>>> c.__dict__["func3_a"] = hook_func3a
>>> c.checkFlag("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuv")
16706 20168
18248 43336
7807 47262
57832 41927
49576 49905
41513 32082
```

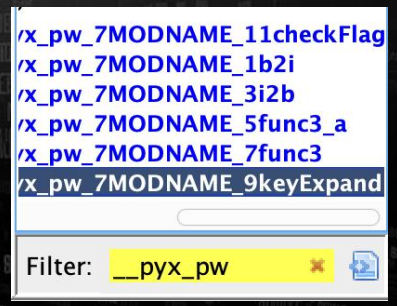
通过静态分析来解题

>>> 静态分析是在动态交互测试**无法完全摸透函数逻辑**的情况下才用的方法，一般用以摸清细节。

. . . 初接触时会觉得很复杂，摸熟了以后就很快上手了。

>>> 全局代码一般位于 **__pyx_pymod_exec_MODNAME**（MODNAME 为该模块的名字）通常其中还包括一些常量的初始化、if main、全局变量和函数的定义等。 #如果不知道主要逻辑在哪个函数里一般就逆这个函数

>>> 定义的函数一般以 **__pyx_pw_*** 的形式，直接在函数表里搜需要的函数即可：



通过静态分析来解题

>>> 静态分析.**难点** = **Cython 包装的框架代码** + **Python 代码的一对多** + **特别长的反编译结果**

>>> 静态分析.**关键点** = **排除无用的框架代码** + **(熟悉一些常见连招, 分块逐个击破)** + **细心**

... # 连招可以通过自己编一个 Cython 扩展、比对反编译结果和 C 代码，来进一步熟悉。

>>> 无用（不影响函数逻辑）的框架代码包括：

... Error Handle

... 引用计数的维护

... 对某些 API 调用的 N 重保障

>>> 警惕函数套娃：

... # 高版本的 Cython 会上 wrapper，这里的 param_2 就是实际调用的参数对象

```
local_58._8_8_ = &__pyx_n_s_out_idx;
local_58._0_8_ = &__pyx_n_s_out;
local_48 = &__pyx_n_s_key;
local_40 = 0;
if (param_4 == 0) {
  if (param_3 == 5) {
    lVar12 = param_2[3];
    lVar21 = param_2[2];
    lVar13 = param_2[1];
    lVar17 = param_2[4];
    lVar20 = *param_2;
    lStack_90 = lVar10;
AB_001129cd:
    uVar14 = __pyx_pf_7MODNAME_6func3.isra.0(lVar20,lVar13,lVar21,lVar12,lVar17);
    return uVar14;
  }
  goto switchD_001129a7_caseD_6;
}
plVar1 = param_2 + param_3;
switch(param_3) {
case 0:
  lVar9 = *(long *)(param_4 + 0x10);
  lVar12 = param_3;
  if (0 < lVar9) {
    do {
```

通过静态分析来解题 - C by Cython

```
__Pyx_GetModuleGlobalName(__pyx_t_2, __pyx_n_s_b2i); if (unlikely(!__pyx_t_2)) __PYX_ERR(0
__Pyx_GOTREF(__pyx_t_2);
__pyx_t_3 = NULL;
__pyx_t_4 = 0;
#if CYTHON_UNPACK_METHODS
if (unlikely(PyMethod_Check(__pyx_t_2))) {
  __pyx_t_3 = PyMethod_GET_SELF(__pyx_t_2);
  if (likely(__pyx_t_3)) {
    PyObject* function = PyMethod_GET_FUNCTION(__pyx_t_2);
    __Pyx_INCREF(__pyx_t_3);
    __Pyx_INCREF(function);
    __Pyx_DECREF_SET(__pyx_t_2, function);
    __pyx_t_4 = 1;
  }
}
#endif
{
  PyObject *__pyx_callargs[3] = {__pyx_t_3, __pyx_v_inp, __pyx_v_inp_idx};
  __pyx_t_1 = __Pyx_PyObject_FastCall(__pyx_t_2, __pyx_callargs+1-__pyx_t_4, 2+__pyx_t_4);
  __Pyx_XDECREF(__pyx_t_3); __pyx_t_3 = 0;
  if (unlikely(!__pyx_t_1)) __PYX_ERR(0, 21, __pyx_L1_error)
  __Pyx_GOTREF(__pyx_t_1);
  __Pyx_DECREF(__pyx_t_2); __pyx_t_2 = 0;
}
__pyx_v_x = __pyx_t_1;
__pyx_t_1 = 0;
```

__pyx_t_2 = b2i

__pyx_t_1 = __pyx_t_2(__pyx_v_inp, __pyx_v_inp_idx)

__pyx_v_x = __pyx_t_1

x = b2i(inp, inp_idx)

通过静态分析来解题 - 反编译 by Ghidra

```
  if (*(long *)(__pyx_d + 0x18) == __pyx_dict_version.22) {
    if (__pyx_dict_cached_value.21 != (long *)0x0) goto LAB_0010e79e;
    plVar5 = (long *)__Pyx_GetBuiltinName(__pyx_n_s_b2i);
LAB_0010f86f:
    if (plVar5 == (long *)0x0) goto LAB_0010f878;
  }
  else {
    __pyx_dict_cached_value.21 =
         (long *)_PyDict_GetItem_KnownHash
                           (__pyx_d,__pyx_n_s_b2i,*(undefined8 *)(__pyx_n_s_b2i + 0
    __pyx_dict_version.22 = *(long *)(__pyx_d + 0x18);
    if (__pyx_dict_cached_value.21 == (long *)0x0) {
      lVar19 = PyErr_Occurred();
      if (lVar19 == 0) {
        plVar5 = (long *)__Pyx_GetBuiltinName(lVar1);
        goto LAB_0010f86f;
      }
LAB_0010f878:
      local_a0 = (long *)0xf2e;
      plVar8 = (long *)0x0;
      plVar12 = (long *)0x0;
      plVar5 = (long *)0x0;
      local_98 = 0x15;
      plVar17 = (long *)0x0;
      plVar6 = (long *)0x0;
      plVar4 = (long *)0x0;
      local_b0 = (long *)0x0;
      local_b8 = (long *)0x0;
      local_c0 = (long *)0x0;
      local_d8 = (long *)0x0;
      local_a8 = (long *)0x0;
      local_d0 = (long *)0x0;
      local_e0 = (long *)0x0;
      local_c8 = (long *)0x0;
      local_e8 = (long *)0x0;
      goto LAB_0010f56f;
```

plVar5 = b2i

x = b2i(inp, inp_idx)

(Error Handle)

__pyx_n_s_* 是通过工具处理后的符号，原反编译结果中没有

通过静态分析来解题 - 反编译 by Ghidra

```
puVar18 = (undefined *)plVar5[1];
plVar8 = (long *)0x0;
uVar20 = 2;
pplVar21 = local_58 + 1;
plVar10 = plVar5;
if ((puVar18 == &PyMethod_Type) && (plVar8 = (long *)plVar5[3], plVar8 != (long *
  plVar10 = (long *)plVar5[2];
  *plVar8 = *plVar8 + 1;
  *plVar10 = *plVar10 + 1;
  *plVar5 = *plVar5 + -1;
  if (*plVar5 == 0) {
    _Py_Dealloc(plVar5,pplVar21,2);
  }
  puVar18 = (undefined *)plVar10[1];
  uVar20 = 3;
  pplVar21 = local_58;
}
local_58[0] = plVar8;
local_58[1] = param_1;
local_58[2] = param_2;
if (((puVar18[0xa9] & 8) == 0) ||
   (*(code **)((long)plVar10 + *(long *)(puVar18 + 0x38)) == (code *)0x0)) {
  plVar4 = (long *)PyObject_VectorcallDict(plVar10,pplVar21,uVar20,0);
}
else {
  plVar4 = (long *)(**(code **)((long)plVar10 + *(long *)(puVar18 + 0x38)))();
}
if ((plVar8 != (long *)0x0) && (*plVar8 = *plVar8 + -1, *plVar8 == 0)) {
  _Py_Dealloc(plVar8);
}
if (plVar4 == (long *)0x0) {
  local_b0 = (long *)0x0;
  plVar8 = (long *)0x0;
  plVar12 = (long *)0x0;
```

(维护引用计数 && Error Handle)

plVar4 = b2i(param_1, param_2)

(Error Handle)

x = b2i(inp, inp_idx)

通过静态分析来解题 - 反编译 by Ghidra

```
    (_DAT_00118048 =
          PyTuple_Pack(0x13,__pyx_n_s_inp,__pyx_n_s_inp_idx,__pyx_n_s_out,__pyx_n_s_out_idx,
                       __pyx_n_s_key,__pyx_n_s_x,__pyx_n_s_y,__pyx_n_s_xh,__pyx_n_s_xl,
                       __pyx_n_s_yh,__pyx_n_s_yl,__pyx_n_s_idx,__pyx_n_s_i,__pyx_n_s_r1,
                       __pyx_n_s_r2,__pyx_n_s_r3,__pyx_n_s_r4,__pyx_n_s_r5,__pyx_n_s_r6),
     _DAT_00118048 == 0))))) ||
  (DAT_00118078 = PyCode_NewWithPosOnlyArgs
                    (5,0,0,0x13,0,3,DAT_00117b40,DAT_00117b38,DAT_00117b38,
                     _DAT_00118048,DAT_00117b38,DAT_00117b38,__pyx_n_s_MODNAME.py,
                     __pyx_n_s_func3,0x14,DAT_00117b40), DAT_00118078 == 0)) ||
```

```
plVar6 = (long *)__Pyx_CyFunction_New.constprop.0
                   (&__pyx_mdef_7MODNAME_7func3,__pyx_n_s_func3,__pyx_n_s_MODNAME,
                    __pyx_d,DAT_00118078);
if (plVar6 == (long *)0x0) {
  plVar10 = (long *)0x0;
  uVar18 = 0x17ef;
  uVar16 = 0x14;
  bVar3 = true;
  goto LAB_0010638c;
}
iVar4 = PyDict_SetItem(__pyx_d,__pyx_n_s_func3,plVar6);
```

`__pyx_n_s_*` 是通过工具处理后的符号，原反编译结果中没有

```
x = b2i(inp, inp_idx)
```

通过静态分析来解题的一些常见小连招

>>> 数字之间的运算通常调用 **PyNumber_*** ，少部分会直接用 C 的运算符

```
plVar13 = (long *)PyNumber_Or(plVar12,local_80);
local_e8 = (long *)PyNumber_And(plVar4,DAT_00118018);
plVar11 = (long *)PyNumber_Add(local_c8,plVar10);
plVar8 = (long *)PyNumber_Xor(local_d0,plVar11);
```

>>> 对如 **int.from_bytes(...)** 之类的调用会用 **GetAttr**

```
plVar8 = (long *)PyObject_GetAttr(&PyLong_Type,DAT_00117bd0)
```

>>> 对字符串的初始化，通常在 **__Pyx_CreateStringTabAndInitStrings** 中

```
// {&__pyx_n_s_from_bytes, __pyx_k_from_bytes, sizeof(__pyx_k_from_bytes), 0, 0, 1, 1},
  local_680 = &DAT_00117bd0; // &__pyx_n_s_from_bytes
  pcStack_678 = "from_bytes"; // __pyx_k_from_bytes
```

>>> 对数字和元组等常量的初始化，通常在 **__pyx_pymod_exec_MODNAME** 中

```
DAT_00118018 = PyLong_FromLong(0xffff), DAT_00118018 == 0

_DAT_00118048 =
    PyTuple_Pack(0x13,DAT_00117c10,DAT_00117c18,DAT_00117c68,DAT_00117c78,DAT_00117c28
                 ,DAT_00117cd8,DAT_00117cf8,DAT_00117ce0,DAT_00117ce8,DAT_00117d00,
                 DAT_00117d08,DAT_00117c00,DAT_00117bf0,DAT_00117c88,DAT_00117c90,
                 DAT_00117c98,DAT_00117ca0,DAT_00117ca8,DAT_00117cb0)
```

通过静态分析来解题的一些常见小连招

>>> **for** 或者 **while** 一出，大部分都是原 Python 代码里的循环（为了优化会不调用 **range** ）

```
local_60 = 8;
do {
  // ...
  local_60 = local_60 + -1;
  if (local_60 == 0) goto LAB_0010fd4d;
  // ...
} while( true );
```

>>> 调用函数一般流程：获取函数 PyCodeObject，把参数放进一个数组里，然后调用 Py*CallDict

```
  plVar10 = (long *)__Pyx_GetBuiltinName(__pyx_n_s_func3_a);
  plVar7 = (long *)PyNumber_And(plVar5,__pyx_int_65535);
    plVar8 = (long *)__Pyx_PyObject_GetItem_Slow(param_5,local_a0);
local_58[0] = plVar12;
local_58[1] = plVar7;
local_58[2] = plVar8;
  local_d0 = (long *)PyObject_VectorcallDict(plVar10,pplVar21,uVar20,0);
```

Cython 逆向之符号恢复

什么？这题居然不给符号？

不能忘记的关键数据结构

根据 Cython / CPython 源码猜出 API

Cython 逆向的噩梦：我的符号呢？

>>> 从放水到步入正常难度（与其他类型的逆向题横向比较）

>>> 难点：

... 找不到函数（比如最重要的 __pyx_pymod_exec_MODNAME 和 __pyx_pw_* ）

... 一些 Cython 中重新实现的函数也无名字了

>>> 突破点：

... 需要熟悉特定结构体

... 唯一的依靠就是 Cpython 的 API

>>> 起手难，上手后难度同有符号表的

寻找函数

>>> 万物起源：PyInit_MODNAME

```
void PyInit_MODNAME(void)

{
  PyModuleDef_Init(&DAT_00117840);
  return;
}
```

. . . Cython 的 methods 多放在 __pyx_moduledef_slots 里

```
static struct PyModuleDef __pyx_moduledef =
#endif
{
    PyModuleDef_HEAD_INIT,
    "MODNAME",
    0, /* m_doc */
  #if CYTHON_PEP489_MULTI_PHASE_INIT
    0, /* m_size */
  #elif CYTHON_USE_MODULE_STATE
    sizeof(__pyx_mstate), /* m_size */
  #else
    -1, /* m_size */
  #endif
    __pyx_methods /* m_methods */,
  #if CYTHON_PEP489_MULTI_PHASE_INIT
    __pyx_moduledef_slots, /* m_slots */
  #else
```

```
static PyModuleDef_Slot __pyx_moduledef_slots[] = {
  {Py_mod_create, (void*)__pyx_pymod_create},
  {Py_mod_exec, (void*)__pyx_pymod_exec_MODNAME},
  {0, NULL}
};
```

```
                        DAT_001178c0
                            ??          01h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                10          addr        FUN_00103679
                00
                            ??          02h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                            ??          00h
                10          addr        FUN_001048fe
                00
```

```
PyMODINIT_FUNC
PyInit_spam(void)
{
    return PyModule_Create(&spammodule);
}
```

```
static struct PyModuleDef spammodule = {
    PyModuleDef_HEAD_INIT,
    "spam",   /* name of module */
    spam_doc, /* module documentation, ma
    -1,       /* size of per-interpreter
                 or -1 if the module keep
    SpamMethods
};
```

```
static PyMethodDef SpamMethods[] = {
    ...
    {"system",  spam_system, METH_VARARGS,
     "Execute a shell command."},
    ...
    {NULL, NULL, 0, NULL}        /* Sentinel */
};
```

寻找函数

>>> 通过 __pyx_pymod_exec_MOD 找到各个函数的定义位置

```
plVar6 = (long *)__Pyx_CyFunction_New.constprop.0
                           (__pyx_mdef_7MODNAME_7func3,DAT_00117bd8,DAT_00117b60,
                            __pyx_mstate_global_static,DAT_00118078);
if (plVar6 == (long *)0x0) {
  plVar10 = (long *)0x0;
  uVar18 = 0x17ef;
  uVar16 = 0x14;
  bVar3 = true;
  goto LAB_0010638c;
}
iVar4 = PyDict_SetItem(__pyx_mstate_global_static,DAT_00117bd8,plVar6);
```

```
  __pyx_t_2 = __Pyx_CyFunction_New(&__pyx_mdef_7MODNAME_7func3, 0, __
  __Pyx_GOTREF(__pyx_t_2);
  if (PyDict_SetItem(__pyx_d, __pyx_n_s_func3, __pyx_t_2) < 0) __PYX_
```

```
static PyMethodDef __pyx_mdef_7MODNAME_7func3 = {
  "func3",
  (PyCFunction)(void*)(__Pyx_PyCFunction_FastCallWithKeywords)__pyx_pw_7MODNAME_7func3,
  __Pyx_METH_FASTCALL|METH_KEYWORDS,
  0
};
```

```
  }
  plVar8 = (long *)FUN_001037c8(&PTR_s_func3_00117940,DAT_00117bd8,DAT_00117b60,DAT_00117b20,
                                DAT_00118078);
  if (plVar8 == (long *)0x0) {
    plVar10 = (long *)0x0;
    uVar17 = 0x17ef;
    uVar15 = 0x14;
    bVar3 = true;
    goto LAB_0010638c;
  }
  iVar4 = PyDict_SetItem(DAT_00117b20,DAT_00117bd8,plVar8);
  if (iVar4 < 0) {
```

```
                    PTR_s_func3_00117940
b2 41 11            addr        s_func3_001141aa+8
00 00 00
00 00
00 29 11            addr        FUN_00112900
00 00 00
00 00
82                  ??          82h
00                  ??          00h
```

```
425       }
426       *plVar8 = *plVar8 + -1;
427       if (*plVar8 == 0) {
428         _Py_Dealloc(plVar8);
429       }
430       plVar8 = (long *)FUN_001037c8(&PTR_s_func3_00117940,D
431                                     DAT_00118078);
432       if (plVar8 == (long *)0x0) {
433         plVar10 = (long *)0x0;
434         uVar17 = 0x17ef;
```

API 符号恢复

>>> 找字符串，搜 github，比对

```
  if (uVar1 != 4) {
1038bf:
    PyErr_SetString(_PyExc_SystemError,"Bad call flags for CyFunction");
    *plVar2 = *plVar2 + -1;
    if (*plVar2 == 0) {
      _Py_Dealloc(plVar2);
```

repo:cython/cython "Bad call flags for CyFunction"

1 file (38 ms) in cython/cython

Cython/Utility/CythonFunction.c

```
679         default:
680             PyErr_SetString(PyExc_SystemError, "Bad call flags for CyFunction");
681             Py_DECREF(op);
842         default:
843             PyErr_SetString(PyExc_SystemError, "Bad call flags for CyFunction");
844             return NULL;
```

```
static PyObject *__Pyx_CyFunction_New(PyMethodDef *ml, int flags, PyObj
                                      PyObject *closure, PyObject *modu
    PyObject *op = __Pyx_CyFunction_Init(
        PyObject_GC_New(__pyx_CyFunctionObject, __pyx_CyFunctionType),
        ml, flags, qualname, closure, module, globals, code
    );
    if (likely(op)) {
        PyObject_GC_Track(op);
    }
    return op;
}
```

__Pyx_CyFunction_Init

Cython 逆向之自动化辅助工具

关键开发思路

开发中的挑战

实战效果

\\cythonHelper//

>>> https://github.com/c10udlnk/cythonHelper

>>> 主要功能 = 提取文件信息 + 全局变量命名 + 跟踪临时变量

>>> 依据 Ghidra 的 P-code 进行插件编写，其余反编译软件插件思路同理。

>>> 一些开发中的挑战

. . . 排除 Cython 框架代码的干扰；

. . . Const 和 RAM 的判断：在实际代参时都为数字，但 P-code 中需要区分，如 **(const, 0x1140fd, 8)**；

. . . Op 的常见顺序和一般编程顺序不同：

```
(unique, 0xc280, 8) LOAD (const, 0x1b1, 4) (unique, 0x3100, 8)
(unique, 0x3100, 8) CAST (unique, 0x10000054, 8)
```

开发思路

>>> 抓住 P-code 中的特定 Opcode，如 **CALL**。

>>> 通过跟踪全局变量赋值，对变量进行命名。

>>> 利用 Ghidra 中的 **getDef()** 和 **getDescendants()** 跟踪数据流。

>>> 将当前 Op 整理成嵌套数组，通过递归调用找到最终的值。

```
(register, 0x0, 8) CALL (ram, 0x103580, 8) (const, 0x9, 8) (ram, 0x117c08, 8) (ram, 0x117c70, 8) (ram,
0x117bf0, 8) (ram, 0x117cd8, 8) (ram, 0x117cf8, 8) (ram, 0x117cf0, 8) (ram, 0x117ba0, 8) (ram, 0x117c00,
8) (ram, 0x117b70, 8)
(register, 0x0, 8) CALL (ram, 0x103590, 8) (const, 0x2, 8) (const, 0x0, 8) (const, 0x0, 8) (const, 0x9,
8) (const, 0x0, 8) (const, 0x3, 8) (ram, 0x117b40, 8) (ram, 0x117b38, 8) (ram, 0x117b38, 8) (register,
0x0, 8) (ram, 0x117b38, 8) (ram, 0x117b38, 8) (ram, 0x117b68, 8) (ram, 0x117c30, 8) (const, 0x30, 8) (
ram, 0x117b40, 8)
(ram, 0x118080, 8) INDIRECT (register, 0x0, 8) (const, 0xc6e, 4)
```

```
{
    00118080
    CALL
    PyCode_NewWithPosOnlyArgs
    const:00000002
    const:00000000
    const:00000000
    const:00000009
    const:00000000
    const:00000003
    00117b40
    00117b38
    00117b38
    {
        00118050
        CALL
        PyTuple_Pack
        const:00000009
        00117c08
        00117c70
        00117bf0
        00117cd8
        00117cf8
        00117cf0
        00117ba0
        00117c00
        00117b70
    }
    00117b38
    00117b38
    00117b68
    00117c30
    const:00000030
    00117b40
}
```

实战效果

```
[+] Get .data(001172e0 - 001179bf), .rodata(00114000 - 00114aff), .bss(001179c0 - 001180b7)
[+] Module name: MODNAME
[+] Get __pyx_moduledef addr: 00117840
[+] Get (Py_mod_create)__pyx_pymod_create addr: 00103679
[+] Get (Py_mod_exec)__pyx_pymod_exec_MODNAME addr: 001048fe
[+] Get StringTabs from "__Pyx_StringTabEntry".
[+] Rename vars completed.
```

```
    (__pyx_int_15 = (long *)PyLong_FromLong(0xf), __pyx_int_15 == (long *)0x0)) ||
   ((__pyx_int_16 = PyLong_FromLong(0x10), __pyx_int_16 == 0 ||
    ((((__pyx_int_22 = PyLong_FromLong(0x16), __pyx_int_22 == 0 ||
       (__pyx_int_33 = (long *)PyLong_FromLong(0x21), __pyx_int_33 == (long *)0x0)) ||
      ((__pyx_int_38 = (long *)PyLong_FromLong(0x26), __pyx_int_38 == (long *)0x0 ||
       (((__pyx_int_39 = (long *)PyLong_FromLong(0x27), __pyx_int_39 == (long *)0x0 ||
         (__pyx_int_46 = PyLong_FromLong(0x2e), __pyx_int_46 == 0)) ||
        (__pyx_int_50 = (long *)PyLong_FromLong(0x32), __pyx_int_50 == (long *)0x0)))))) |
     ((__pyx_int_51 = (long *)PyLong_FromLong(0x33), __pyx_int_51 == (long *)0x0 ||
```

```
(___pyx_tuple_8_items_s_master_key_dst_key_out_i____ =
      PyTuple_Pack(8,__pyx_n_s_s,__pyx_n_s_master_key,__pyx_n_s_dst,__pyx_n_s_key,
                   __pyx_n_s_out,__pyx_n_s_i,__pyx_n_s__,__pyx_n_s__,uVar16),
___pyx_tuple_8_items_s_master_key_dst_key_out_i____ == 0)) ||
(__pyx_codeobj_checkFlag =
     PyCode_NewWithPosOnlyArgs
               (1,0,0,8,0,3,__pyx__bytes_,__pyx_tuple_0_items,__pyx_tuple_0_items,
                ___pyx_tuple_8_items_s_master_key_dst_key_out_i____,__pyx_tuple_0_items,
                __pyx_tuple_0_items,__pyx_n_s_MODNAME.py,__pyx_n_s_checkFlag,0x41,
                __pyx__bytes_), __pyx_codeobj_checkFlag == 0)) goto LAB_001065aa;
```

Future...

>>> 提取特定函数的输入输出以帮助重建函数逻辑

```
local_58 = (long *)&__pyx_n_s_b;
plVar8 = (long *)PyObject_GetAttr(&PyLong_Type,__pyx_n_s_from_bytes);
pplVar17 = &local_58;
plVar9 = (long *)PyObject_VectorcallDict(plVar8,pplVar17,uVar16,0);
```

→

```
plVar9 = PyLong_Type.from_bytes(b)
```

>>> 提取数组

```
plVar6 = (long *)PyList_New(0x30);
plVar11 = __pyx_int_194;
pplVar2 = (long **)plVar6[3];
*pplVar2 = plVar11;
plVar11 = __pyx_int_39;
pplVar2[1] = plVar11;
plVar11 = __pyx_int_14;
pplVar2[2] = plVar11;
plVar11 = __pyx_int_214;
pplVar2[3] = plVar11;
...
```

→

```
plVar6 = [194, 39, 14, 214, ...]
```


CTFCON
2024
THANK YOU